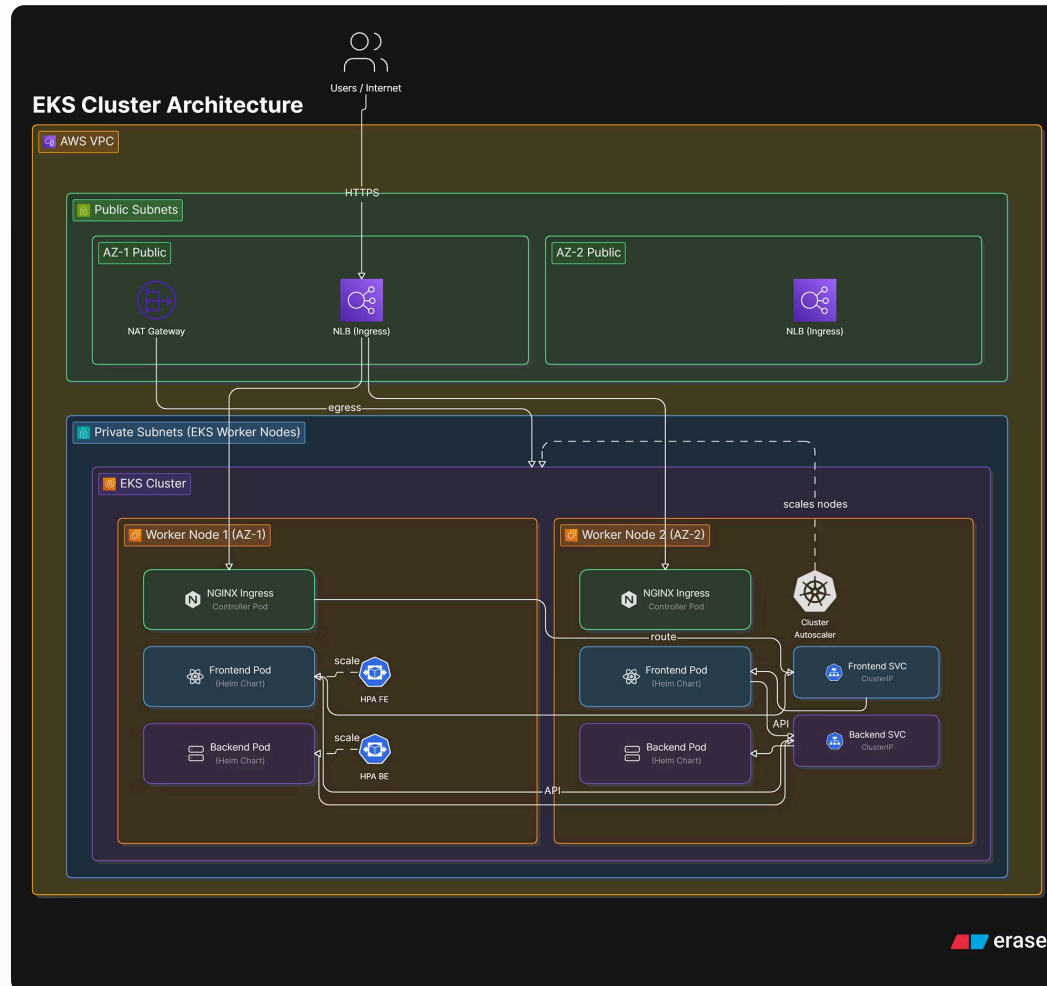


Platform Architecture



Infrastructure Layers

- **VPC** — Public & private subnets, route tables, NAT Gateway, Security Groups
- **EKS Cluster** — Managed control plane with Cluster Autoscaler
- **Observability** — Prometheus, Grafana, EFK Stack (Elasticsearch, Fluent Bit, Kibana)
- **Ingress** — NGINX Ingress Controller for external traffic routing
- **App Layer** — Node.js microservice with HPA, rolling updates, health probes

Technology Stack



Cloud & IaC

AWS EKS, VPC, Terraform modules (official + CloudDrove IAM)



Kubernetes

Cluster Autoscaler, NGINX Ingress, HPA, Rolling Updates



Logging

Elasticsearch, Fluent Bit, Kibana — full EFK stack



Containerization

Docker, Docker Hub, Helm charts for app & add-ons



Monitoring

Prometheus, Grafana, ServiceMonitor auto-discovery



CI/CD

GitHub Actions — validate, build, deploy, verify

GitHub Actions Pipeline

CI validates every pull request before merge. CD automatically promotes approved changes to the cluster — with rollout verification and live smoke tests ensuring zero-downtime deployments.

CI Pipeline

1. Trigger on PR

GitHub Actions starts on every pull request to catch issues early before code reaches main.

3. Validate Terraform

Run format, validate, and plan checks to confirm infrastructure changes are safe and reproducible.

5. Build Docker image

Package the application into a versioned image for release consistency.

7. Upload artifacts

Store manifests, build outputs, and reports for downstream release steps.

CD Pipeline

1. Trigger on merge to main

A merge to main starts the release pipeline automatically for approved changes.

2. Build & Push Docker Image

Build the Docker image from the artifact source and push the versioned image to Docker Hub for deployment.

3. Create EKS Cluster (terraform apply)

Run terraform apply -auto-approve to provision or update the EKS cluster infrastructure, ensuring all networking, IAM, and node groups are in place.

4. Deploy to EKS via Helm

Use Helm to upgrade the Kubernetes release on Amazon EKS with the approved image tag.

5. Run smoke tests

Perform post-deploy checks to confirm the app responds correctly in the live environment.

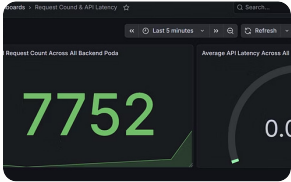
6. Verify rollout status

Wait for the deployment to become healthy and confirm pods are updated successfully.

MONITORING

Grafana Dashboards

Four dashboards provide full-stack visibility — from infrastructure health to API performance.

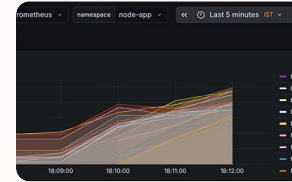


API Latency & Request Count

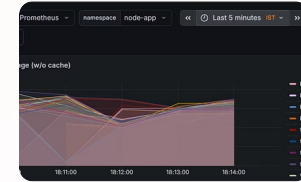


Node CPU / Memory

Cluster node utilization



Pod CPU Usage



Pod Memory Usage

Node memory utilization over time

LOGGING

Kibana Dashboards

The EFK stack surfaces application-level insights — errors, traffic patterns, and endpoint health at a glance.

Error Tracking

Filter by severity, timestamp, and pod to isolate failures fast

Request Volume

Aggregate request counts with time-series breakdowns

HTTP Status Codes

Distribution of 2xx, 4xx, and 5xx responses across endpoints

Top Endpoints

Ranked view of highest-traffic routes for capacity planning

Key Learnings

IRSA via Pod Identity Association

`aws_eks_pod_identity_association` is the modern replacement for IRSA — bind IAM roles to service accounts without legacy annotations.

Helm as the Unified Delivery Format

Packaging Deployments, Services, ServiceMonitors, and HPA into one chart keeps manifests versioned and reproducible.

CI Validation Catches Infra Drift Early

Running `terraform validate` and `kubeconform` on every PR prevents broken plans from reaching the cluster.

Structured JSON Logs Are Non-Negotiable

Fluent Bit parses JSON natively — unstructured logs break field extraction and Kibana filters downstream.

CHALLENGES

Challenges & Fixes

Challenge 1 — Module Version Mismatch

CloudDrove's `iam-role` module (latest) threw a `version.tf` error. Manual edits would break CI, so I pinned `latest - 1` — a stable release that resolved the issue without forking.

Challenge 2 — Cluster Autoscaler CrashLoop

The pod started before `aws_eks_pod_identity_association` was ready, causing permission errors. Adding `depends_on` enforces correct creation order and eliminates the race.

Improvements Roadmap

HTTPS via cert-manager

Auto-provision TLS certificates with Let's Encrypt, eliminate HTTP traffic, and enforce encrypted ingress

1

Secrets Management

Replace Kubernetes Secrets with AWS Secrets Manager or External Secrets Operator for encryption at rest

3

2

Network Policies

Enforce pod-to-pod traffic isolation using Calico or AWS Network Policy Controller for zero-trust networking

4

Policy as Code

Integrate OPA Gatekeeper or Kyverno to enforce security, resource, and compliance policies at admission time

Key Learnings

Implementing this platform provided valuable insights into best practices for cloud infrastructure, security, and operational efficiency.

→ Remote Terraform State

Storing state in S3 enables collaboration, state locking, and recovery instead of relying on local state files.

→ Pod Identity Security

Pod Identity provides a more secure and scalable approach than managing AWS credentials inside Kubernetes workloads.

→ Resource Dependencies Matter

Resource dependencies and deployment order are critical in Infrastructure as Code and can directly impact application availability.

→ Observability is Essential

Combining metrics, logs, and dashboards helps troubleshoot issues faster and improve system reliability.

→ Reusable Infrastructure

Building reusable Terraform modules and Helm charts makes infrastructure and application deployments more maintainable and scalable.